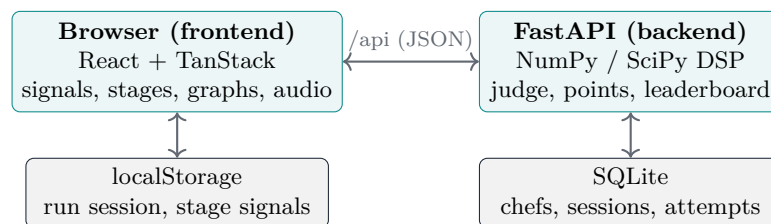

WaveKitchen / WaveBakery

Detailed Architecture

The inner mathematics, the function behind every operation, and the step-by-step workings of generation, filtering, mixing, seasoning, marinating, cooking, the oven, the z-plane, scoring and the leaderboard



Contents

1	System architecture	2
1.1	Processes and responsibilities	2
1.2	Frontend storage (<code>localStorage</code>)	2
1.3	Backend tables (<code>backend/app/models.py</code>)	2
2	Conventions: how a signal is represented	3
2.1	The <code>PipelineSignal</code>	3
2.2	From samples to sound (<code>frontend/src/lib/audio.ts</code>)	3
3	Signal generation	4
3.1	Frontend ingredient models (<code>frontend/src/lib/signals.ts</code>)	4
3.2	Backend ingredient synthesis and contamination	4
4	Filtering	5
4.1	Server filter (authoritative)	5
4.2	Cleaning (prep) score — <code>metrics.prep_quality</code>	5
4.3	Frontend Filtering Lab (<code>routes/filtering.tsx</code>)	5
5	Mixing	6
6	Seasoning, marinating, caramelizing, chopping	6
6.1	Seasoning — <code>computeSeasonedSignal</code>	6
6.2	Marinating — <code>computeMarinatedSignal</code>	6
6.3	Caramelizing — amplitude modulation	7
6.4	Chopping — decimation	7
7	Cooking — convolution	7
7.1	Impulse responses	7
7.2	<code>computeConvolvedSignal</code> (<code>lib/pipeline.ts</code>)	7
8	Precision Oven (<code>frontend/src/lib/precision-oven-dsp.ts</code>)	8
8.1	Maximum frequency and the Nyquist rate	8
8.2	Sampling and aliasing demonstration — <code>analyzeSamplingAndAliasing</code>	8
8.3	Spectrum used for tuning — <code>computeDiscreteSignalFFT</code>	8
8.4	Equaliser — <code>tuneOvenFrequencies</code>	8
8.5	Reconstruction — <code>computeSignalIFFT</code>	8
8.6	Oven score	9
9	System Delivery — the z-plane (<code>frontend/src/lib/z-system.ts</code>)	9
10	Similarity metrics	9
10.1	Frontend (<code>computeSignalSimilarity</code> , <code>frontend/src/lib/dsp.ts</code>)	9

10.2 Server dish score (<code>metrics.dish_metrics</code>)	10
11 Scoring, step by step	10
11.1 During the run: stage accuracies	10
11.2 The reference dish	10
11.3 Server judgement — <code>gameplay.judge</code>	10
11.4 Submission (exactly once)	11
11.5 What the score screen shows (<code>routes/score.tsx</code>)	11
12 Points, ranks, tiers and leaderboards	11
12.1 Career points and ranks (<code>backend/app/gameplay.py</code>)	11
12.2 Server leaderboards (<code>backend/app/routers/leaderboard.py</code>)	11
12.3 On-device leaderboard (<code>frontend/src/lib/leaderboard.ts</code>)	12
13 Security and validation	12
14 Function index	12

1 System architecture

1.1 Processes and responsibilities

Part	Responsibility
<code>run.py</code>	Starts the backend (<code>uvicorn app.main:app --reload</code> , port 8000; on Windows in its own hidden console so its reload Ctrl+C cannot stop the runner) and the Vite dev server (port 5173). Relays backend logs; stops both on exit.
Frontend	Synthesises ingredient signals, runs every station's DSP, draws, plays audio, keeps the run in <code>localStorage</code> , and talks to the backend through <code>frontend/src/lib/api.ts</code> (header <code>X-Player-Token</code>).
Backend	Owns chefs (hashed passwords, tokens), sessions and attempts; regenerates ingredients from a seed; rebuilds and scores the dish; computes points, ranks, tier unlocks and leaderboards.

1.2 Frontend storage (`localStorage`)

Key	Contents
<code>wavebakery_recipe_session</code>	The run: recipe, difficulty, start/end time, every stage accuracy, every dial (seasoning, marinate, appliance, bowl, chop, caramelize, cart), final score/stars, backend session id and result. Type <code>RecipeRunSession</code> in <code>lib/recipes.ts</code> .
<code>wavebakery_progress_<recipe></code>	Highest unlocked level (1–9).
<code>wavebakery_pipeline_<recipe>_<stage></code>	A saved <code>PipelineSignal</code> for stage <code>raw</code> , <code>filtered</code> , <code>mixed</code> , <code>seasoned</code> , <code>marinated</code> , <code>cooked</code> , <code>delivered</code> .
<code>wavebakery_filtered_ingredients_<recipe></code>	Filtered samples per washable ingredient.
<code>wavebakery_cooked_signal_<recipe></code>	The cooked dish (<code>CookedSignalData</code>).
<code>wavebakery_selected_ingredients_<recipe></code>	Ingredients picked in Generate.
<code>wavebakery_leaderboard_entries</code>	The on-device leaderboard.
<code>wavekitchen_player_token</code> , <code>wavebakery_chef_name</code>	Server token and chef name.
<code>wavebakery_recipe_best_scores</code>	Best total per recipe.

Changing a stage calls `invalidateDownstreamStages` (`lib/pipeline.ts`), which deletes every later stage's saved signal so nothing stale is carried forward.

1.3 Backend tables (`backend/app/models.py`)

Table	Fields that matter
players	handle, token, script <code>password_hash</code> , points, unlocked tier.
ingredients, appliances, recipes	Catalogue from <code>seed.py</code> ; recipes hold targets (seasoning, blend, marinate, caramelize carrier/depth, chop factor, appliances), tier, tolerance, noise difficulty.
game_sessions	One run: seed, status (<code>active/served/abandoned</code>), difficulty, params (JSON).
session_ingredients	Per-slot seed, contaminants, filter bands/tools, prep score, accepted.
attempts	A served dish: score, stars, prep score, SNR, MSE, correlation, spectral similarity, points, difficulty, notes. The leaderboard's source.

2 Conventions: how a signal is represented

2.1 The PipelineSignal

Every frontend stage produces

$$\mathbf{x} = (x[0], \dots, x[N-1]), \quad N = 401, \quad t_n = \frac{n}{N-1} \in [0, 1],$$

plus metadata (frequency label, duration label, and optionally the 2-D mixing curve). A frequency of f “game Hz” means f cycles in the window.

Helper functions (`frontend/src/lib/pipeline.ts`)

- **sampleAt($\mathbf{x}, u$)**: periodic linear interpolation. With $u' = u \bmod 1$, $p = u'(N-1)$, $i = \lfloor p \rfloor$:
 $x(u) = x[i] + (p - i)(x[i+1] - x[i])$.
- **resampleSignal($\mathbf{x}, M$)**: $y[m] = \text{sampleAt}(\mathbf{x}, m/(M-1))$.
- **normalizeSamples($\mathbf{x}, 0.95$)**: scales *down only*: $y = x \cdot \min(1, 0.95/\max_n |x[n]|)$.

2.2 From samples to sound (`frontend/src/lib/audio.ts`)

SignalAudioPlayer treats the window as one period-group and loops it. With the signal's frequency label f (default 4),

$$f_0 = \text{clip}\left(220 \cdot \frac{f}{4}, 110, 880\right) \text{ Hz}, \quad \text{windows per second} = \frac{f_0}{\max(1, f)}.$$

Output sample k at the device rate F_s ($t = k/F_s$) is $s(t) = x((t \cdot f_0/f) \bmod 1) \cdot g \cdot e(t)$ with gain $g = 0.75/\max |x|$ and a 30 ms attack / 120 ms release envelope $e(t)$. So the audible fundamental is f_0 and the timbre is exactly the drawn waveform. The recorded chicken plays its decoded WAV buffer directly.

Loop seam (`smoothLoopSeam`): looping jumps from $x[N-1]$ back to $x[0]$. If that jump is larger than $1.5\times$ the largest step inside the signal (a signal that does not complete whole cycles in the window, e.g. carrot, milk or a delayed dish), the last 5 % of the window is blended towards $x[0]$ with a raised-cosine ramp $x[n] + (x[0] - x[N-1])\frac{1}{2}(1 - \cos \pi u)$, removing the click that would otherwise repeat at the loop rate. Genuine edges (square waves) are left untouched.

3 Signal generation

3.1 Frontend ingredient models (`frontend/src/lib/signals.ts`)

Each ingredient is an `IngredientSignalDefinition` with `evaluate`, `generateSamples` and, for 2-D shapes, a `parametricCurve`. The router `getRecipeIngredientSamples` (`lib/pipeline.ts`) returns 401 samples:

1. **Chicken** → `generateChickenSamples`: linear interpolation of the decoded PCM (`lib/chicken-samples.ts`) over the whole 2.16 s clip.
2. **Catalogued ingredient** → its `generateSamples( $f, A, \phi$ , noise)`.
3. **Fallback**: $x[n] = A \sin(2\pi f t_n + \phi)$.

Periodic families

$$\text{triangle (Cheese): } x(t) = A \frac{2}{\pi} \arcsin(\sin(2\pi f t + \phi)) = \frac{8A}{\pi^2} \sum_{k \text{ odd}} \frac{(-1)^{(k-1)/2}}{k^2} \sin(2\pi k f t + \dots)$$

$$\text{square (Sugar, Salt): } x(t) = A \operatorname{sgn}(\sin(2\pi f t + \phi)) = \frac{4A}{\pi} \sum_{k \text{ odd}} \frac{1}{k} \sin(2\pi k f t + \dots)$$

The Fourier series explains what the Filtering and Oven spectra show: a square wave keeps strong odd harmonics ($1/k$), a triangle loses them quickly ($1/k^2$).

Parametric ingredients and their 1-D signal

A curve $\gamma(\theta) = (X(\theta), Y(\theta))$, $\theta \in [\theta_{\min}, \theta_{\max}]$, is sampled at 601 points for drawing. The 1-D signal uses the vertical component over the same parameter range, e.g. Noodles ($\theta \in [-12\pi, 12\pi]$):

$$x[n] = A \cos(-12\pi + 24\pi t_n + \phi) \quad (\text{i.e. } Y/5).$$

Frontend noise model (`ingredientNoise`)

Five tones above the ingredient's band (15.3, 18.7, 22.1, 26.9, 31.4 game Hz):

$$n(t) = 0.85 \nu \frac{1}{5} \sum_{i=1}^5 \sin(2\pi f_i t + 37.1 \phi_s i), \quad x_{\text{dirty}} = x - n.$$

Tones are used instead of a hashed pseudo-random sequence so the noise is genuinely spread above the cutoff instead of aliasing into one spike.

3.2 Backend ingredient synthesis and contamination

Backend signals are real audio: $F_s = 22\,050$ Hz, frame length 4096 (≈ 186 ms) (`backend/app/dsp/core.py`). `clean_signal` calls `instruments.synth` (additive partials with ADSR envelopes: piano, guitar, bell, flute, ..., plus mathematical voices for cheese, sugar, salt, bread, patty, carrot, cucumber, sauce, egg).

`contamination.corrupt(clean, difficulty  $d$ , seed)` uses a seeded RNG so the same session always reproduces the same dirty ingredient:

- number of layers = $1 + \operatorname{rand}\{0, \dots, \min(2, \lfloor d \rfloor)\}$;
- each layer's amplitude $a = 0.08 + 0.10 d (0.6 + 0.8u)$, $u \sim U(0, 1)$;

- type chosen from **{white** (whole band), **hiss** (high-passed noise starting just above the ingredient’s ideal cutoff $\times(1.1..1.6)$), **burst** (sparse clicks)};
- $x_{\text{dirty}} = x + \sum_{\ell} n_{\ell}$, and every contaminant’s band $[f_{lo}, f_{hi}]$ is recorded.

4 Filtering

4.1 Server filter (authoritative)

`apply_filter_chain` (`backend/app/dsp/pipeline.py`) is an FFT-mask filter:

$$N = 2^{\lceil \log_2 L \rceil}, \quad X[k] = \text{FFT}_N\{x\}, \quad Y[k] = X[k] H(f_k), \quad f_k = \frac{kF_s}{N}, \quad y = \text{IFFT}_N\{Y\}[0..L-1].$$

The player’s cutoff becomes a low-pass tool with a raised-cosine transition of width $w = \max(30, 0.15f_c)$ (`core.lowpass`):

$$H(f) = \begin{cases} 1 & f \leq f_c \\ 1 - \frac{1}{2}(1 - \cos \pi \frac{f-f_c}{w}) & f_c < f < f_c + w \\ 0 & f \geq f_c + w \end{cases}$$

(Graphic-EQ bands, high-pass, band-pass, notch and a harmonic “de-hum” comb are built the same way and multiplied: `chain.`)

4.2 Cleaning (prep) score — `metrics.prep_quality`

With magnitude spectra C (clean), D (dirty), F (filtered) and the set of noise bins $\mathcal{B}$ (inside any contaminant band):

$$\begin{aligned} \text{noise}_{\text{before}} &= \sum_{k \in \mathcal{B}} \max(0, D_k - C_k), & \text{noise}_{\text{after}} &= \sum_{k \in \mathcal{B}} \max(0, F_k - C_k), \\ \text{removal} &= \text{clip}\left(1 - \frac{\text{noise}_{\text{after}}}{\text{noise}_{\text{before}}}, 0, 1\right), & \text{preservation} &= \text{clip}\left(\frac{\sum_k \min(C_k, F_k)}{\sum_k C_k}, 0, 1\right), \\ \text{prep} &= 100 (0.55 \text{removal} + 0.45 \text{preservation}). \end{aligned}$$

Flags: *over-filtered* if $\text{preservation} < 0.55$; *still dirty* if $\text{removal} < 0.5$. `accept_ingredient` rejects $\text{prep} < 45$ or either flag. It is deliberately not a distance to the clean signal, which would reward doing nothing when the noise is quiet.

4.3 Frontend Filtering Lab (`routes/filtering.tsx`)

1. Input: the server’s dirty/clean plots for the slot when a session exists, otherwise `getRecipeIngredientSamples` with noise 0.85 (dirty) and 0 (clean).
2. Live preview while dragging (cutoff $f_c \in [0, 900]$, ideal f^*): noise = dirty – clean; if $f_c < f^*$: signal gain $\max(0, 1 - \frac{f^* - f_c}{200})$, noise gain 0; else noise gain $(\frac{f_c - f^*}{900 - f^*})^{1.5}$.
3. *Apply*: sends a low-pass tool with that cutoff to the server and shows its filtered plot and prep score. Cleanliness = server prep score (offline: the similarity of filtered vs clean, or $97 - |f_c - f^*|/6$ before applying).
4. Pass mark: cleanliness ≥ 85 . *Advance* accepts the ingredient on the server, saves the filtered samples (`saveFilteredIngredient`) and records the stage accuracy as the running average of cleanliness over the washed ingredients.

5 Mixing

`computeMixedSignal` (`lib/pipeline.ts`)

For bowl ingredients $k = 1..K$ with samples x_k (the saved filtered samples if the ingredient was washed, otherwise its generated samples, noisy if washable):

$$m[n] = \text{normalize}_{0.95} \left(\frac{1}{\sqrt{K}} \sum_{k=1}^K x_k[n] \right), \quad K = 1 : m = x_1 \text{ (pass-through)}.$$

The $1/\sqrt{K}$ factor keeps the RMS of K uncorrelated signals comparable to one signal.

The mixing-bowl curve (`computeSuperpositionCurve`, `withMixCurve`)

If any bowl ingredient is parametric, the bowl is the geometric sum at a common normalised parameter $u \in [0, 1]$ (601 points):

$$X(u) = \sum_{k \in \text{param}} X_k(\theta_k(u)), \quad Y(u) = \sum_{k \in \text{param}} Y_k(\theta_k(u)) + \sum_{j \in 1-D} x_j(u).$$

The curve is attached as metadata (`curve`); the samples $m[n]$ are unchanged, so filtering still reaches the mix. Later stages transform the curve alongside the samples (`transformCurve`, `delayCurve`, `carryCurve`) and `pipelineSignalToPath` draws it.

Accuracy (`handleMix`, `routes/mixing.tsx`): $\text{mix} = 100 \cdot \frac{|\text{bowl} \cap \text{recipe}|}{|\text{recipe}|}$. The bowl's ingredient names are stored in the session and synced to the server.

Server (`bowl_signals`, `stage_mix`): recipe ingredients in the bowl use their filtered signal, extra catalogue ingredients are added clean, missing ones are absent; then $m = \frac{1}{\sqrt{K}} \sum_k \text{resize}(x_k, 4096)$.

6 Seasoning, marinating, caramelizing, chopping

6.1 Seasoning — `computeSeasonedSignal`

$$s[n] = A \cdot m((\alpha t_n) \bmod 1) \quad (\text{via } \text{timeScaleSamples}),$$

i.e. $y(t) = Ax(\alpha t)$ with the mix repeated periodically when $\alpha > 1$. No renormalisation, so A really changes the height. The frequency label becomes αf . The curve: $Y \mapsto Y_{\text{mid}} + A(Y - Y_{\text{mid}})$, parameter sped up by α with an x -offset per repeat. **Accuracy** (`routes/transform.tsx`):

$$\text{acc} = \max \left(0, 100 - 70 \left(\frac{|A - A^*|}{A^*} + \frac{|\alpha - \alpha^*|}{\alpha^*} \right) \right), \quad \text{pass at 85}.$$

Server: `stage_season` $y = Ax$; `stage_blend` $y[n] = x(\alpha n)$ by linear interpolation with zeros past the end (`core.time_scale`); length becomes $\lfloor L/\alpha \rfloor$.

6.2 Marinating — `computeMarinatedSignal`

With time s (seconds) and duration label $D = 3$:

$$d = \text{round} \left(\frac{s}{\max(1, D)} \cdot N \right), \quad r[n] = \begin{cases} m \left(\frac{n-d}{N-1} \right) & 0 \leq n-d \\ 0 & n < d \end{cases}, \quad r \leftarrow \text{normalize}_{0.95}(r).$$

The curve gets a flat lead-in at the centre line (`delayCurve`). **Accuracy**: $\max(0, 100 - 150 \frac{|s - s^*|}{s^*})$, pass at 85. **Server**: `stage_marinate` delays by sF_s samples after zero-padding; integer delays copy

samples, fractional delays use the shift theorem

$$Y[k] = X[k] e^{-j2\pi k d/N} \quad (\text{core.time_shift}).$$

6.3 Caramelizing — amplitude modulation

Frontend (`routes/caramelize.tsx`): $y[n] = r[n](1 + m \cos(2\pi \frac{f_c}{10} t_n))$, then normalised to 0.95; **accuracy** = $\text{clip}(100 - 45 \frac{|f_c - 180|}{180} - 55|m - 0.55|, 10, 100)$. Server (`core.am_modulate`): $y(t) = x(t) [1 + m \cos(2\pi f_c t)]$ with t in seconds. In frequency: $Y(f) = X(f) + \frac{m}{2} X(f - f_c) + \frac{m}{2} X(f + f_c)$ — two sidebands.

6.4 Chopping — decimation

Frontend (`routes/chop.tsx`): zero-order-hold decimation by M , $y[n] = x[M \lfloor n/M \rfloor]$; the effective rate is $22050/M$ and “aliasing” is declared when anti-aliasing is off, $M > 1$ and $800f > 22050/(2M)$, in which case a synthetic tone of $|22050/M - 800f|/100$ cycles per window and amplitude 0.35 is added (a simulation, not true folding). **Accuracy**: $\max(20, 100 - 25|M - 3|)$, minus 35 (floor 15) without anti-aliasing. Server (`core.decimate`): optional low-pass at $0.9 \cdot \frac{F_s}{2M}$, keep every M -th sample, zero-order hold back to the original length; new Nyquist $F_s/(2M)$.

7 Cooking — convolution

7.1 Impulse responses

Frontend kernels (`generateStaticCookingKernel`, 96 taps, $\sum_k |h[k]| = 1$):

$$h_{\text{grill}}[k] = e^{-k/16} (\tau_k + 0.15 \sin(1.85k) e^{-k/16}), \quad \tau_k \text{ taps } 1, .55, .38, .22, .12, .06 \text{ at } k = 0, 4, 11, 21, 35, 54$$

$$h_{\text{fry}}[k] = e^{-k/22} (0.45 \sin(12.99k + 3.14) + 0.35 \sin(31.42k + 1.2) + 0.2 \cos(57.17k))$$

$$h_{\text{bake}}[k] = e^{-((k-16)/14)^2} \quad (\text{Gaussian, a smooth low-pass})$$

$$h_{\text{boil}}[k] = e^{-k/26} (0.7 \sin(\pi k/7) + 0.3 \sin(\pi k/14)) \quad (\text{damped resonator})$$

Server impulse responses (`backend/app/dsp/systems.py`): grill = comb (delay 90, feedback 0.62, 7 taps), fry = short bright decay ($\tau = 130$), bake = long dark decay ($\tau = 700$, low-passed), boil = resonator $e^{-k/\tau} \sin(2\pi f_0 k / F_s)$ at 320 Hz, simmer = 48-tap moving average, sear = $[1, -1]$ (differencer), plus smoke and steam composites.

7.2 computeConvolvedSignal (lib/pipeline.ts)

Slider position $p \in [0, 100]$, $\tau = p/100$, shift $s = \text{round}(30\tau)$:

$$c[n] = \sum_{k=0}^{95} x((n - k - s) \bmod N) h[k] \quad (\text{circular convolution}),$$

$$y[n] = (1 - w) x[n] + w c[n], \quad w = \min(1, 1.15\tau),$$

plus a small crackle $0.04\tau \sin(27.13n + 7.4)$ for frying, then normalised to 0.95. **Accuracy** (`routes/cooking.tsx`): 50 if the recipe’s method is chosen (else 15) + $\min(50, p/2)$. The target cooked signal uses the recipe’s method at $p = 100$.

Server (`stage_cook`, `apply_system`): linear convolution $y = x * h$ (SciPy, direct or FFT), truncated to the input length, rescaled to 0.95 only if it clips. Several appliances are one system: **cascade** $h = h_1 * h_2 * \dots$.

8 Precision Oven (`frontend/src/lib/precision-oven-dsp.ts`)

8.1 Maximum frequency and the Nyquist rate

findMaxSignalFrequency: resample the dish to 128 points, apply a Hann window $w[n] = \frac{1}{2}(1 - \cos \frac{2\pi n}{127})$, FFT, and take the first bin k at which the cumulative energy $\sum_{i=1}^k |X_i|^2$ reaches 95 % of the total (DC excluded); with a 128-point window over one second each bin is 1 game Hz wide, so $f_{\max} = \text{clip}(k, 2, 32)$ game Hz. Minimum safe rate $= 2f_{\max}$.

8.2 Sampling and aliasing demonstration — `analyzeSamplingAndAliasing`

1. Samples $x_m = x(m/f_s)$ for $m = 0..[f_s] - 1$ over the one-second window.
2. Periodic sinc reconstruction over three periods: $\hat{x}(t) = \sum_{p=-1}^1 \sum_m x_m \text{sinc}(f_s(t - \frac{m}{f_s} - p))$.
3. Folding of the five strongest harmonics when $f > f_s/2$: $f_{\text{alias}} = |f - \text{round}(f/f_s)f_s|$ (reflected again if still above $f_s/2$).

8.3 Spectrum used for tuning — `computeDiscreteSignalFFT`

$N_{\text{FFT}} = \max(64, 2^{\lceil \log_2 f_s \rceil})$, $x[m] = x(m/f_s)$ ($m < f_s$), 0 otherwise (zero padding), $f_k = \frac{k f_s}{N_{\text{FFT}}}$.

No window is applied (the edited spectrum is inverted back). Displayed magnitude $|X_k| \cdot 2/f_s$, normalised by 1.25 max. Radix-2 Cooley–Tukey FFT with bit reversal (`cooleyTukeyRadix2`). Bins are classed as Base Warmth [0, 6), Crumb [6, 16), Top Crisp [16, 32] game Hz (`getBandAvailability` marks a band unavailable/partial when $f_s/2$ is below it).

8.4 Equaliser — `tuneOvenFrequencies`

$$X'_k = X_k \cdot G_{\text{band}(f_k)} \cdot C(f_k) \cdot Q(f_k), \quad C(f) = \begin{cases} 1 & f \leq f_c \\ \max(0.01, e^{-((f-f_c)/4)^2}) & f > f_c \end{cases},$$

$$Q(f) = 1 - \frac{0.88}{1 + ((f - f_n)/1.5)^2} \quad \text{for } |f - f_n| \leq 3 \text{ (notch on),}$$

$$X'_{N-k} = \overline{X'_k} \quad (\text{Hermitian symmetry, so the result stays real}).$$

8.5 Reconstruction — `computeSignalIFFT`

IFFT, keep the first $N_s = [f_s]$ samples, then band-limited periodic interpolation with the Dirichlet kernel back to 401 points:

$$\hat{x}(t) = \sum_{m=0}^{N_s-1} x'_m D_{N_s}\left(t - \frac{m}{N_s}\right), \quad D_N(\tau) = \begin{cases} \frac{\sin(\pi N \tau)}{N \sin(\pi \tau)} & N \text{ odd} \\ \frac{\sin(\pi N \tau)}{N \tan(\pi \tau)} & N \text{ even} \end{cases}$$

then normalised to 0.95. With untouched controls and $f_s \geq 2f_{\max}$ this reproduces the dish (verified $\geq 98\%$ similarity by `frontend/tests/precision-oven-fft.test.ts`).

8.6 Oven score

$$\begin{aligned} \text{spectrum match} &= \text{clip}\left(100\left(1 - \frac{\sum_k |c_k - t_k|}{1.5 \sum_k t_k}\right), 0, 100\right) \quad (\text{computeSpectrumSimilarity}) \\ \text{waveform fidelity} &= \text{computeSignalSimilarity}(\hat{x}, \text{target}) \quad (\text{Section 10}) \\ \text{efficiency} &= \begin{cases} 100 & f_s \leq 1.25 \cdot 2f_{\max} \\ 100\left(1 - \frac{f_s - 2.5f_{\max}}{64 - 2.5f_{\max}}\right)^+ & \text{otherwise} \end{cases} \\ \text{oven} &= 0.40 \text{ spectrum} + 0.50 \text{ fidelity} + 0.10 \text{ efficiency}. \end{aligned}$$

The unlock step rejects $f_s < 2f_{\max}$ and $f_s > 64$. *Plate* saves the reconstruction as the **delivered** stage and records the oven score as the delivery accuracy.

9 System Delivery — the z-plane (`frontend/src/lib/z-system.ts`)

Preset	$H(z)$	Poles / zeros / gain
1st-order low-pass	$\frac{(1-r)z}{z-r};$ $y[n] = (1-r)x[n] + ry[n-1]$	pole r , zero 0, $g = 1 - r$ (unity DC gain)
2nd-order resonator	$\frac{z^2}{(z - re^{j\omega_0})(z - re^{-j\omega_0})}$	conjugate poles, double zero at 0, $g = 1$; $y[n] = x[n] + 2r \cos \omega_0 y[n-1] - r^2 y[n-2]$
Moving average	$\frac{1}{6} \frac{1 - z^{-6}}{1 - z^{-1}}$	zeros $e^{j2\pi k/6}$, $k = 1..5$; five poles at 0; $g = 1/6$ (FIR)
Notch	$\frac{(z - e^{j\omega_0})(z - e^{-j\omega_0})}{(z - 0.85e^{j\omega_0})(z - 0.85e^{-j\omega_0})}$	zeros on the unit circle, poles at radius 0.85

Frequency response and filtering

- Magnitude on the unit circle, geometrically (`SystemResponsePlotter.tsx`): $|H(e^{j\omega})| = g \frac{\prod_i |e^{j\omega} - z_i|}{\prod_j |e^{j\omega} - p_j|}$ (clipped to 4), phase = $\sum \arg(e^{j\omega} - z_i) - \sum \arg(e^{j\omega} - p_j)$, 128 points on $[0, \pi]$.
- Dish spectrum: 64-point DFT of the dish; bin $k \leq 32$ sits at $\omega_k = 2\pi k/64$ and is multiplied by $|H(e^{j\omega_k})|$ (an unstable cart shows $\times 5$).
- Actual filtering (`applySystem`): expand $\prod(z - p_j)$ and $g \prod(z - z_i)$ into real coefficients a , b (`expandRoots`), then run the difference equation $y[n] = \sum_k b_k x[n - k] - \sum_{k \geq 1} a_k y[n - k]$. This plays as “Listen equalized dish”.

Score (`systemAccuracy`; server `system_score_for` computes the same from the synced settings): 100 if $\max_j |p_j| < 1$ (BIBO stable) else 15, multiplied by 0.6 when the sensor rate is below 4000 Hz.

10 Similarity metrics

10.1 Frontend (`computeSignalSimilarity`, `frontend/src/lib/dsp.ts`)

The player’s samples are resampled to the target length, then

$$r_{xy} = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sqrt{\sum (x - \bar{x})^2 \sum (y - \bar{y})^2}}, \quad \text{NRMSE}_{\text{sim}} = \text{clip}\left(1 - \frac{\sqrt{\frac{1}{N} \sum (x - y)^2}}{\max y - \min y}, 0, 1\right),$$

$$\text{similarity} = 100(0.5 \max(0, r_{xy}) + 0.5 \text{NRMSE}_{\text{sim}}).$$

Correlation rewards the right *shape*; NRMSE rewards the right *level*.

10.2 Server dish score (`metrics.dish_metrics`)

Both dishes are normalised to peak 0.9; then

$$\begin{aligned} \text{SNR} &= 10 \log_{10} \frac{\sum t^2}{\sum (t-p)^2} \text{ dB}, & \text{corr} &= \max_{\ell} \frac{(t \star p)[\ell]}{\|t\| \|p\|} \quad (\text{peak of the cross-correlation}), \\ \text{spec} &= \frac{\langle T_{\text{dB}}, P_{\text{dB}} \rangle}{\|T_{\text{dB}}\| \|P_{\text{dB}}\|}, & T_{\text{dB}} &= \max(20 \log_{10} |T|, -80) + 80, \\ \text{dish} &= 100 \left(0.40 \text{clip} \frac{\text{spec} - 0.75}{0.25} + 0.35 \text{clip}(\text{corr}) + 0.25 \text{clip} \frac{\text{SNR} + 5}{30} \right) \quad (\text{each clip to } [0, 1]). \end{aligned}$$

11 Scoring, step by step

11.1 During the run: stage accuracies

Each station calls `recordStageAccuracy (lib/recipes.ts)`, which clamps to $[0, 100]$ and stores it on the run session: filtering, mixing, seasoning, marinating, cooking, chop, caramelize, delivery (oven), system (cart). Dials are stored too and pushed to the server by `syncSessionParamsToBackend`. The payload fields are `seasoning`, `frequency`, `blend`, `marinate`, `appliances`, `bowl`, `carrier`, `depth`, `chop_factor`, `anti_alias`, `delivery_accuracy`, `system_preset`, `system_pole_radius` and `system_sampling_hz`.

11.2 The reference dish

Frontend `getIdealDishSignal`: clean mix of all recipe ingredients (`computeCleanMixedSignal`) $\rightarrow$ seasoning with $(A^*, \alpha^*) \rightarrow$ marinating with $s^* \rightarrow$ convolution with the recipe method at $p = 100$.
Server judge: `run_pipeline(clean ingredient signals, reference_params(recipe))` — the same function that builds the player’s dish.

11.3 Server judgement — `gameplay.judge`

1. Rebuild the player’s dish: regenerate each ingredient from its seed, apply the stored filters, keep only the bowl, run `run_pipeline` with the stored params: mix $\rightarrow$ season (A) $\rightarrow$ blend (α) $\rightarrow$ marinate (s) $\rightarrow$ caramelize (f_c, m) $\rightarrow$ chop (M) $\rightarrow$ cook (appliances) $\rightarrow$ normalise 0.9.
2. $m = \text{dish_metrics}(\text{reference final}, \text{player final})$.
3. Prep average $\bar{P}$ over washable/contaminated ingredients (all ingredients if none).
4. Sub-scores: filtering = $\bar{P}$; mixing = dish score of the two mixes; transformation = dish score of the two marinated signals; cooking = dish score of (player’s pre-cooking signal through the recipe’s appliances) vs (through the player’s).
5. $\text{raw} = 0.72 m_{\text{score}} + 0.28 \bar{P}$.
6. Finishing stations, when present: $\text{raw} \leftarrow \text{raw} (1 - W) + \sum w_i v_i$ with $(v, w) = (\text{oven}, 0.10)$ and $(\text{cart}, 0.05)$, $W = \sum w_i$.
7. Tolerance curve: $\text{score} = 100 (\text{raw}/100)^\gamma$, $\gamma = 1/\max(0.3, \text{tolerance})$ (tolerance 0.75 for the Feast up to 1.25 for Toast: harder recipes are graded more strictly).

8. Stars: $\geq 92 \rightarrow 5$, $\geq 80 \rightarrow 4$, $\geq 66 \rightarrow 3$, $\geq 50 \rightarrow 2$, $\geq 32 \rightarrow 1$, else 0.
9. Notes (`_diagnose`): named mistakes from the parameter errors (over/under-seasoned, blended, marinated; wrong AM carrier or depth; wrong chop factor; aliasing; wrong appliance chain; over-filtered or still-dirty ingredients).

11.4 Submission (exactly once)

`submitRunToBackend` (`lib/recipes.ts`) syncs the params, then `POST /sessions/{id}/submit`; concurrent callers share one in-flight request. The server claims the session atomically (`UPDATE ... WHERE status != 'served'`); a second submit gets 409; if judging fails the claim is released. A served session can no longer be edited.

11.5 What the score screen shows (`routes/score.tsx`)

The player dish is the delivered dish if the oven was used, otherwise the cooked dish. The stage average is the mean of filtering, mixing, seasoning, marinating and cooking, plus chop and caramelize for recipes that include those stations.

```

similarity = computeSignalSimilarity(player dish, reference dish)
local = 0.5 similarity + 0.5 stageAvg   (then the finishing-station blend)
score100 = server score if available, otherwise local
base = round(10 · score100 · μdiff),    μ ∈ {0.8, 1.0, 1.25, 1.5}
time bonus = round(min(2 · seconds left, 300) · μdiff)
total = base + time bonus.
```

A stage with no recorded accuracy counts as 0 (except filtering on a recipe with nothing to wash, which counts 100). Stars use the server's stars or the same thresholds on `score100`. The total, stars and similarity are stored on the session; the Complete page (`routes/complete.tsx`) only displays them.

12 Points, ranks, tiers and leaderboards

12.1 Career points and ranks (`backend/app/gameplay.py`)

- **award**: if the new score beats the chef's best on that recipe, `points = round(score × tier × 1.5)` is added; otherwise 0. Runs on a tier the chef has not unlocked earn 0 and do not count for unlocking.
- **rank_for**: Dishwasher (0), Prep Cook (250), Line Cook (600), Sous Chef (1100), Head Chef (1800), Signal Master (2600).
- **maybe_unlock**: when the chef has cleared `min(2, #recipes)` distinct recipes of their current tier with score ≥ 60 , the next tier opens.

12.2 Server leaderboards (`backend/app/routers/leaderboard.py`)

`GET /leaderboard?recipe_id=&difficulty=&limit=`

All attempts ordered by score (then time), de-duplicated to each chef's best, then cut to `limit`. With a difficulty, attempts of that difficulty plus older attempts that have none recorded.

`GET /leaderboard/global`

Chefs ordered by career points, then best score; with rank title and dishes served.

GET /stats

Players, sessions started, dishes served, average score, and per recipe: plays, average, best and five-star rate.

12.3 On-device leaderboard (frontend/src/lib/leaderboard.ts)

The score screen calls `addLeaderboardEntry` with `id run-<recipe>-<difficulty>-<start>`: one entry per chef, recipe and difficulty, keeping the higher score. `getLeaderboard` filters by recipe and difficulty and sorts by score. The leaderboard page shows the server's rows when it has any, otherwise these; it also re-submits a finished run that never reached the server (backend offline) and shows a retry notice.

13 Security and validation

- Passwords: scrypt ($N = 2^{14}$, $r = 8$, $p = 1$, 16-byte salt), constant-time comparison (`backend/app/security.py`). A chef created before passwords existed gets one on first sign-in.
- Every session endpoint checks the token owns the session (`owned_session`).
- DSP endpoints bound every input (sample rate 1–96 kHz, $\leq 200\,000$ finite samples, ≤ 4096 -tap custom impulse, 1–64 speakers).
- The SPA file route only serves files inside `frontend/dist`; CORS allows only the frontend's origins.
- The server never trusts client audio: it regenerates ingredients from the seed and rebuilds the dish. The one client-reported number is the Precision Oven score.

14 Function index

Function	File	Mathematical role
Frontend		
<code>evaluate*Wave, generate*Samples</code>	<code>lib/signals.ts</code>	Ingredient signal models
<code>ingredientNoise</code>	<code>lib/signals.ts</code>	Additive multi-tone noise
<code>computeSuperpositionCurve</code>	<code>lib/signals.ts</code>	2-D geometric superposition
<code>getRecipeIngredientSamples</code>	<code>lib/pipeline.ts</code>	Ingredient $\rightarrow$ 401 samples
<code>sampleAt, resampleSignal, normalizeSamples</code>	<code>lib/pipeline.ts</code>	Interpolation, resampling, peak limiting
<code>computeMixedSignal</code>	<code>lib/pipeline.ts</code>	Superposition with $1/\sqrt{K}$
<code>withMixCurve</code>	<code>lib/pipeline.ts</code>	Attach bowl curve
<code>computeSeasonedSignal, timeScaleSamples</code>	<code>lib/pipeline.ts</code>	$Ax(\alpha t)$
<code>computeMarinatedSignal, delayCurve</code>	<code>lib/pipeline.ts</code>	$x(t - t_0)$
<code>computeConvolvedSignal</code>	<code>lib/pipeline.ts</code>	Circular convolution, wet/dry
<code>carryCurve, transformCurve, pipelineSignalToPath</code>	<code>lib/pipeline.ts</code>	Curve follows each stage (display)
<code>getIdealDishSignal, computeCleanMixedSignal</code>	<code>lib/pipeline.ts</code>	Reference dish
<code>generateStaticCookingKernel</code>	<code>lib/cooking-audio.ts</code>	Impulse responses $h[n]$

Function	File	Mathematical role
fft, applyLowPassFilter, convolve	lib/dsp.ts	FFT, filtering, convolution
normalizedCrossCorrelation, normalizedRootMeanSquareError, computeSignalSimilarity	lib/dsp.ts	Similarity
findMaxSignalFrequency	lib/precision-oven-dsp.ts	95 %-energy bandwidth
analyzeSamplingAndAliasing	lib/precision-oven-dsp.ts	Sampling, sinc reconstruction, folding
computeDiscreteSignalFFT, cooleyTukeyRadix2	lib/precision-oven-dsp.ts	Zero-padded DFT / FFT
tuneOvenFrequencies	lib/precision-oven-dsp.ts	Band gains, cutoff, notch
computeSignalIFFT	lib/precision-oven-dsp.ts	IFFT + Dirichlet reconstruction
computeSpectrumSimilarity	lib/precision-oven-dsp.ts	Spectral match
systemPolesZeros, applySystem, systemAccuracy	lib/z-system.ts	$H(z)$, difference equation, stability
SignalAudioPlayer	lib/audio.ts	Sonification
recordStageAccuracy, syncSessionParamsToBackend, submitRunToBackend	lib/recipes.ts	Accuracies, sync, submission
addLeaderboardEntry, getLeaderboard	lib/leaderboard.ts	Local leaderboard
Backend		
fft, ifft, cooley_tukey_fft	dsp/core.py	Transforms
lowpass, highpass, bandpass, notch, equalizer, chain	dsp/core.py	Frequency responses
build_mask, apply_mask, filter_signal	dsp/core.py	FFT-mask filtering
time_scale, time_shift	dsp/core.py	$x(\alpha t)$, $x(t - t_0)$
convolve, apply_system, cascade, frequency_response	dsp/core.py	LTI systems
am_modulate, decimate	dsp/core.py	AM, down-sampling
corrupt	dsp/contamination.py	Seeded noise
synth	dsp/instruments.py	Ingredient audio
ir_comb, ir_decay, ir_resonator, ir_moving_average, ir_difference	dsp/systems.py	Appliance $h[n]$
run_pipeline, reference_params, apply_filter_chain	dsp/pipeline.py	The canonical dish pipeline
prep_quality, dish_metrics, snr_db, spectral_similarity, stars	dsp/metrics.py	Scores

Function	File	Mathematical role
<code>bowl_signals</code> , <code>compute_stages</code> , <code>judge</code> , <code>award</code> , <code>maybe_unlock</code> , <code>system_score_for</code>	<code>gameplay.py</code>	Judging and progression